
Compiler-Guided Autoformalization with Adaptive Multi-View Representations

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Autoformalization requires both faithful interpretation and a computational route
2 for reasoning about it. We describe a compiler-guided architecture coupling en-
3 gineered symbolic IRs to adaptive multi-view representation learning. Determin-
4 istic parsers construct source-grounded declarations; decompilers expose source-
5 withheld reconstruction errors; and shared feature, family, semantic-slot, and view
6 parameters are optimized by an implemented packed tensor update and guarded
7 outer selection. BM25, vector indexes, and knowledge-graph traversal supply
8 complementary context rather than truth criteria. A policy-constrained Tactician
9 organizes search, an ITP Hammer translates supported fragments and reconstructs
10 solver candidates, and Leanstral supplies advisory proof-engineering proposals. In-
11 dependently admitted feedback trains isolated proof heads or motivates separately
12 tested compiler/decompiler changes. Stable learned guidance reaches enabled
13 consumers only through source-free, fixed-canary promotion. Shared symbols
14 and explicit preservation contracts connect deontic, temporal, frame, event, graph,
15 Legal, Security, and Intent views without equating their semantics. We distinguish
16 numerical reconstruction from symbolic round trips, and model prediction from
17 checked proof. A synthetic policy-code-trace example and twelve standalone
18 semantic checks illustrate composition. This source-audited methods draft reports
19 no new checkpoint training or end-to-end performance: held-out source fidelity,
20 retrieval benefit, native proof coverage, and total learning/verification cost remain
21 to be measured.

22 1 Introduction

23 Autoformalization makes heterogeneous material available to mechanical reasoning, but a formula
24 can be well typed and provable while misrepresenting its source. A requirement, implementation, and
25 execution trace also have different truth conditions: an obligation is not an observation, permission
26 is not execution, and belief is not fact. Recent mathematical autoformalization demonstrates useful
27 natural-language translation into formal statements [1, 2, 9]. Our question is how to learn better
28 source-to-symbolic mappings while preserving these distinctions across domains and logical views.

29 The method couples a deterministic semantic pipeline to adaptive representation learning. Compilers
30 create source-grounded intermediate representations (IRs), decomposers expose multiple logical
31 views, and decompilers reconstruct inspectable content. An adaptive modal autoencoder learns
32 reusable feature-to-embedding and feature-to-family/view parameters against these targets. Recon-
33 struction, structural, provenance, grammar, and cross-view diagnostics identify where the symbolic
34 pipeline is inadequate. Bounded optimization then either updates model parameters or proposes a
35 separately validated compiler/decompiler improvement. Checked proof feedback supplies additional,
36 isolated prediction heads rather than rewarding arbitrary changes that make the input easier to prove.

37 This is neither one end-to-end language model nor an unsupervised discovery of a universal logic.
 38 The schema, operators, sorts, and admissibility rules are engineered. Learning estimates content,
 39 associations, rankings, and repair priorities within those contracts. Several paths coexist: determin-
 40 istic compilation, adaptive diagnostic learning, genuine tensor gradient updates, source-withheld
 41 reconstruction, and bounded model-assisted synthesis. We distinguish their inputs, outputs, objectives,
 42 and activation instead of describing all of them as one autoencoder.

43 The contribution of this methods draft is: (i) a source-grounded account of representation learn-
 44 ing and compiler/decompiler improvement; (ii) an explicit account of BM25, vector, and graph
 45 retrieval, policy-constrained proof planning, Hammer reconstruction, and Leanstrat guidance; and
 46 (iii) composition of complementary IR views through typed identities and justified translations. A
 47 worked policy-code-trace example and finite semantic witnesses illustrate the necessary boundaries.
 48 The final learned-fidelity, transfer, checkpoint, and cost experiments remain marked for completion.
 49 Source inspection establishes implementation mechanisms, not successful training of every optional
 50 configuration.

51 **2 Representations and training examples**

52 **2.1 What is shared, and what is learned?**

53 Three axes must remain distinct. *Source format* names prose, code, traces, tables, or extracted media.
 54 *Domain* names Legal, Security, Intent, software, or UI declarations. *Logical view* names deontic,
 55 temporal, frame, first-order, cognitive/event, graph, or proof-oriented structure. Sharing canonical
 56 records does not equate domain ontologies or logical interpretations. [artifact S01–S06]

57 The domain-neutral kernel supplies canonical bytes, identities, provenance, source spans, assumptions,
 58 claims, obligations, and result protocols. Domain adapters retain their own semantic slots. The syntax
 59 kernel supplies typed terms, binders, signatures, connectives, and versioned family extensions. A
 60 symbolic declaration is therefore discrete and inspectable; its CID identifies an encoding under a
 61 declared profile, not a learned location in a vector space. Equal vectors or nearby embeddings do not
 62 imply equivalent formulas. [artifact S01,S05]

63 Six intermediate objects have different roles: canonical symbolic declarations, family-specific
 64 formulas, learned numerical features, graph relationships, search plans, and proof/evidence records.
 65 A search plan is not a formula; a declaration is not its verification result; and a vector is not a proof.
 66 Their interchange requires explicit mappings. A model can select or repair a candidate without
 67 defining its semantics. [artifact S01,S04,S34,S38]

68 **2.2 How examples and supervisory targets are constructed**

69 The concrete legal training record contains source identity and citation, raw and normalized text,
 70 an embedding-model identifier and vector, a modal IR document, frame candidates and selection,
 71 parser trace, and losses. A builder normalizes a U.S.-Code-style passage, parses modal formulas,
 72 ranks ontology frames using BM25, and packages the result. The default convenience embedding is
 73 explicitly a deterministic mock; the local codec can instead supply spaCy-derived feature hashing,
 74 and externally supplied vectors carry their producer identity. Experiments must record which path
 75 supplied the target vector. [artifact S22,S23]

76 Thus targets are not all human gold labels. Modal-family targets can be derived from parsed operators;
 77 view targets come from bridge contracts and their configured scoring; frames can be retrieval-selected;
 78 and proof-feedback labels require admitted verifier records. Reproducing these labels measures
 79 distillation of those teachers. Independent annotations and held-out semantic distinctions are required
 80 to measure fidelity to the original source.

81 For a source x , write $I = C_\psi(x)$ for the symbolic compiler output, $e = T(x)$ for the declared embed-
 82 ding target, and $f = F(x, I)$ for reusable features. Features include lexical cues, actor/action/object
 83 roles, conditions and exceptions, temporal structure, predicate arity, quantifier scope, provenance,
 84 frame relations, compiler contracts, and reconstruction diagnostics. The adaptive model predicts an
 85 embedding $\hat{e}$, a modal-family distribution p_F , and a view distribution p_V . Some features depend on
 86 an existing parse, so this is often *parser-assisted learning*, not a demonstrated raw-text-only universal
 87 compiler. [artifact S24]

88 3 Compilers, decompilers, and reconstruction loops

89 3.1 The deterministic source-to-symbolic path

90 The local modal codec uses deterministic registry cues, spaCy-derived features, BM25 frame ground-
91 ing, and frame-logic consistency checks. It produces a structured modal declaration before optional
92 expensive model repair. A separate selected structured-text profile has a narrower seven-field rule:
93 modality, actor, action, object, conditions, exceptions, and temporal qualifiers. Its modality vo-
94 cabulary is obligation, permission, and prohibition; its atom vocabulary is supplied by the caller.
95 Unsupported semantics trigger abstention or disclosed partial output. The selected profile explicitly
96 disables learned guidance and repair: training elsewhere in the repository does not imply that this
97 frozen profile silently uses it. [artifact S07,S08,S23,S25]

98 Formalization adapters produce named formula views and source maps. The maps bind IR constituents
99 to source evidence; they are not proof of a correct interpretation. Source-code extraction offers
100 additional candidate security facts, with explicit heuristic and coverage limitations. Media requires a
101 separately measured extraction layer before this logic pipeline; no end-to-end audio/image theorem
102 performance is established here. [artifact S09,S13–S16]

103 3.2 Two different meanings of reconstruction

104 Symbolic reconstruction follows

$$I_1 = C_{\psi}(x), \quad \hat{x} = D_{\omega}(I_1), \quad I_2 = C_{\psi}(\hat{x}). \quad (1)$$

105 A decompiler renders roles, predicates, modality, conditions, exceptions, and temporal structure
106 into inspectable text. The canonical source-withheld evaluation gives the realizer the IR, not the
107 originating source, its locator, source map, private parser state, or gold target. Comparing I_1 and I_2
108 tests reconstructable facets and compiler/realizer consistency. It does not prove source fidelity: both
109 programs could consistently omit the same exception. [artifact S07]

110 The adaptive autoencoder’s numerical reconstruction is different. Its encode method builds a state
111 containing family predictions and an embedding projection; its decode returns that projection as
112 a numeric vector. Its basic reconstruction target is the sample embedding, not arbitrary source
113 prose. Textual decompilation and structural cycle metrics enter through the codec and multi-view
114 bridge. Calling this vector reconstruction a trained free-text decoder would misstate the inspected
115 implementation. [artifact S24]

116 3.3 Constrained candidates and transfer beyond the legal teacher

117 The constrained decoder gives learned heads a closed production space: valid-token, grammar, binder,
118 type, and family masks precede candidate scoring. The inspected profile declares a frozen 143-token
119 vocabulary and bounded search, but these declarations are not a learned tokenizer checkpoint or
120 evidence of end-to-end sequence training. The learner may rank admitted productions; a deterministic
121 grammar decides what can be represented. [artifact S33]

122 The original sample builder is legal-specific. A domain-neutral formalization/advisor layer permits
123 other domain adapters to share feature, view, checkpoint, and result contracts, but it does not
124 automatically turn a legal-trained model into a calibrated Security or Intent formalizer. Cross-domain
125 use needs explicit adapter mappings and separate evaluation. Shared infrastructure, statistical transfer,
126 and semantic proof transfer are three distinct claims. [artifact S01,S17,S22]

127 4 Adaptive multi-view learning

128 4.1 Shared parameter state versus per-example memory

129 The adaptive implementation has sparse parameter groups for feature embeddings, modal families,
130 semantic slots, predicate arguments, logic signatures, view logits, decompiler plans, and family–slot–
131 view interactions. It also retains compatibility fields indexed by sample identity. The generalizable
132 training route disables per-sample memory for updates and evaluation; accepted improvements must

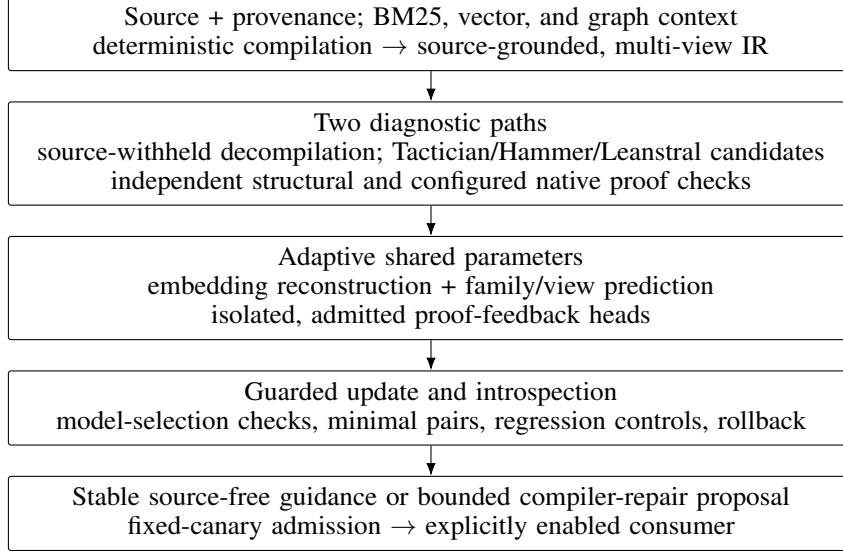


Figure 1: Coupled compilation, retrieval, proof assistance, and learning. Arrows show permissible information flow, not one automatically activated entry point. Only admitted feedback trains proof heads; the frozen canonical compiler remains deterministic.

133 come from shared feature parameters. This distinction is essential because reconstructing a stored
134 vector for a known sample is not generalization. [artifact S24]

135 A schematic description of the shared heads is

$$\hat{e} = e_0(f) + \sum_g A_g(f)W_g, \quad (2)$$

$$p_F = \text{softmax}\left(\ell_F^0(f) + \sum_g A_g(f)U_g\right), \quad (3)$$

$$p_V = \text{softmax}\left(\ell_V^0(f) + \sum_g A_g(f)V_g\right). \quad (4)$$

136 Here A_g is the activity of a bounded feature group, and W_g, U_g, V_g stand for enabled embedding and
137 logit tables. This notation summarizes the packed implementation, not a claim that it is a Transformer
138 or variational autoencoder. Some groups may be disabled, and a forward-only no-op is reported
139 separately from a parameter update. [artifact S26–S28]

140 4.2 Actual gradient objectives

141 The packed tensor implementation gathers active rows, constructs predictions, reduces losses in
142 FP32, backpropagates, clips gradients, applies an SGD-style step, and scatters changed rows into
143 transaction-aware state. BF16 activations are optional after a numerical check; the same packed
144 path has a CPU reference. This is real differentiation through tensor losses, not just the word “SGD”
145 applied to a prose critique loop. The inspected update is not an Adam update. [artifact S26–S28]

146 For the active heads, its objective is

$$L_{\text{packed}} = H(q_F, p_F) + H(q_V, p_V) + \text{MSE}(e, \hat{e}) + \lambda_c(1 - \cos(e, \hat{e})) + L_{\text{norm}} + \lambda_2 R(\Theta). \quad (5)$$

147 Targets are normalized distributions; masks exclude unavailable target rows. Reconstruction is
148 averaged over embedding coordinates, cosine loss guards zero-norm vectors, the auxiliary term
149 penalizes extreme norm ratios, and regularization is normalized over parameters. Terms for inactive
150 update targets are zero. The outer controller additionally scores structural/view losses and regression
151 constraints; it is not simply Equation 5 applied to every available metric. [artifact S27]

152 Family targets may be mixed. Evaluation reports cross-entropy, target entropy, and excess cross-
153 entropy. Since $H(q, p) - H(q) = D_{\text{KL}}(q \| p)$, a nonzero cross-entropy need not be avoidable error:
154 it can include the target distribution’s own entropy. Neither cosine similarity nor cross-entropy is a
155 theorem about recovered meaning. [artifact S24]

156 4.3 Guarded optimization rather than an unrestricted fit

157 The generalizable projection trainer selects update groups—family logits, decoded embeddings,
158 global or feature-specific view logits, and combined updates—and evaluates candidate learning-rate
159 steps. It can concentrate on hard examples, use bounded pre-screening, and select a candidate only
160 when the chosen objective improves without disallowed regression. Candidate changes are evaluated
161 transactionally and rolled back before selection; timeout, no-improvement, and regression remain
162 visible. A configurable deadband can relax a named numerical tolerance, so its mode and protected
163 metrics belong in the experiment manifest. [artifact S24]

164 Validation examples supplied to this repeated selection procedure are *model-selection data*, not an
165 untouched test set. Moreover, when no validation set is supplied, the inspected function falls back
166 to evaluating training samples. The paper’s generalization claims therefore require explicit disjoint
167 source-family splits, separate final test data, disabled sample memory, and frozen targets. Source
168 derivatives and multiple views of one passage must not cross partitions.

169 4.4 Proof-aware auxiliary supervision

170 Seven explicit heads predict obligation family, semantic slots, premise family, route availability,
171 trusted outcome, reconstruction success, and minimal failing contract. Categorical heads use nor-
172 malized class probabilities; multilabel heads use independent sigmoid probabilities. Only version-
173 matched, trusted, eligible feedback trains them; duplicates and excluded holdout records are tracked.
174 The code reports coverage, abstention, and family-specific metrics. [artifact S24]

175 Critically, these proof-head parameters are isolated. Their declared weight in the primary representa-
176 tion objective is zero, and protected compiler cross-entropy, embedding, structural reconstruction,
177 provenance, and anti-copy parameter fingerprints are checked before and after an update. A prediction
178 of proof success can prioritize a route but cannot certify a formula. This protects against learning to
179 replace a difficult source statement with an easier, different theorem. [artifact S24]

180 4.5 How learning improves deterministic formalization

181 Introspection exposes feature contributions, embedding residuals, family margins, underrepresented
182 view components, and pipeline-stage diagnostics. The TODO supervisor separates parameter updates
183 from deterministic program-synthesis tasks. A high reconstruction loss can request a shared-weight
184 update; a missing exception-scoping rule requires a compiler/decompiler repair and new regression
185 tests. The cost-gated loop avoids expensive model repair unless it is enabled and its configured
186 expected benefit exceeds call cost. Those estimates are decisions to evaluate, not measured economic
187 guarantees. [artifact S24,S25,S29]

188 Stable-feature promotion is an explicit join between the two loops. The export excludes raw source,
189 token-level copying channels, sample identifiers, and sample-memory state. Integration resolves it
190 against canonical view contracts and requires paired, fixed-canary evidence, including source-copy
191 and symbolic-validity guardrails, before emitting content-addressed guidance with rollback metadata.
192 This is a guarded transfer of learned regularities into a deterministic compiler-facing interface, not
193 gradient descent through arbitrary parser code and not unconstrained rule invention. [artifact S30]

194 5 Retrieval, tacticians, and proof reconstruction

195 5.1 Three retrieval signals, not three truth criteria

196 BM25 supplies lexical candidates: the sample builder ranks frames, while a separate GraphRAG thin
197 client reads sharded postings and applies term-frequency, inverse-frequency, and length-normalization
198 scoring. Vector indexes instead retrieve neighbors of a declared embedding; the inspected FAISS
199 store maintains indexes, metadata, and identifier mappings. Graph adjacency and bounded walks

200 expand typed relationships and dependencies. These signals answer different questions—matching
201 terms, numerical similarity, and explicit connectivity—and none establishes entailment. [artifact
202 S22,S34,S35]

203 Index construction is not encoder training. Nor does a common result envelope establish that every
204 entry point supports every mode: the inspected thin-client CLI declares a vector option but does not
205 dispatch it to an implemented vector query. The actual producer, backend, revision, and route must
206 be recorded. Context found by any index becomes a premise only after grounding, applicability, and
207 policy checks; absence from retrieval is not logical negation. Appendix N details these boundaries.

208 5.2 The Tactician plans; the Hammer searches and reconstructs

209 The repository’s `LogicTactician` deterministically orders caller-provided sources under an explicit
210 policy, records selected/excluded routes, creates an acyclic subgoal plan, and reports stop or abstention
211 conditions. Planning performs no proof, network work, or writes. Optional model guidance can
212 reorder or nominate admitted sources under a pinned digest, with deterministic fallback. This planner
213 is not itself a trained tactic synthesizer. [artifact S38]

214 The ITP Hammer selects premises, lowers a supported first-order-safe fragment to TPTP or SMT-
215 LIB, runs an allowed bounded solver portfolio, and attempts native reconstruction. Vampire/E are
216 first-order automation routes; `Z3/cvc5` are SMT routes; `Lean`, `Coq/Rocq`, and `Isabelle/HOL` have
217 separate frontend/reconstruction contracts. A logic family specifies what a statement means; a prover
218 family specifies a computation over an admitted encoding. Unsupported dependent, higher-order, or
219 unresolved polymorphic constructs cannot be silently erased. External execution is opt-in, and native
220 availability must be established. [artifact S37]

221 Within this Hammer contract, solver output is a candidate until reconstruction is accepted by the
222 originating ITP’s checker in its pinned environment. The deterministic premise baseline uses symbol,
223 type, import, and graph overlap, not a universal vector ranker. Its optional graph-selector artifact uses
224 hand-authored linear weights; the inspected release does not ship a genuinely trained premise-selector
225 checkpoint. Ranking machinery, training a ranker, and proving a goal are separate claims. [artifact
226 S36,S37]

227 5.3 Leanstral proposals and verified training feedback

228 Leanstral supplies advisory proof-engineering candidates, such as Lean edits or strategies. The
229 security lane requires the suggested proof to pass the Lean checker and rejects proof bypasses; a
230 model response does not replace source, runtime, or release evidence. The legal rollout separately
231 admits structured guidance through syntax, provenance, anti-copy, proof, and configured Lean-
232 reconstruction checks. Raw drafts never become reconstruction targets or trusted training labels.
233 Verification creates a new structured artifact rather than making the original prose authoritative.
234 [artifact S39,S40]

235 Together, retrieval nominates context, the Tactician organizes bounded work, Hammer and Leanstral
236 provide different proposal mechanisms, and checking determines the supported result. Admitted feed-
237 back can train isolated heads or identify a compiler defect; a versioned patch still needs independent
238 regression tests and rollout approval. This is not evidence that Leanstral’s base weights are fine-tuned
239 by the adaptive autoencoder. The concrete contracts and return path are given in Appendix O.

240 6 How different IRs are used together

241 6.1 Semantics are indexed by a profile

242 Each logical view has a signature, sentence language, semantic models, satisfaction relation, and a
243 supported query fragment. The common record layer does not force one implementation of these
244 components. A computable representation is not a universal decision procedure: queries may require
245 decidable fragments, bounded search, or explicit unknown/timeout outcomes. First-order and frame
246 views express objects and relations; event-calculus views express events, fluents, and change [6];
247 deontic views express normative force; temporal views bind trace/clock semantics; cognitive views
248 distinguish agent-indexed knowledge, belief, and intention. World, agent, time, action, and formula
249 categories cannot be exchanged merely because their identifiers are strings.

250 For illustration, a normal modality can be interpreted with an indexed accessibility relation:

$$\llbracket \Box_i \phi \rrbracket(w) = \forall v [R_i(w, v) \Rightarrow \llbracket \phi \rrbracket(v)]. \quad (6)$$

251 Different R_i and frame axioms express different profiles. A reflexive epistemic relation supports
 252 factivity; a serial belief relation need not. A deontic ideal-world relation need not make the obligation
 253 true in the actual world. This is an illustrative semantic recipe, not a claim that every repository
 254 view is already compiled by Equation 6. Conditional norms, priorities, defeasible exceptions, and
 255 contrary-to-duty obligations require their own explicit semantics; ordinary normal modal logic is not
 256 a universal deontic solution. LogiKEY provides established semantic embeddings of deontic logics
 257 and combinations in HOL [3].

258 **The repository’s principal view families.** TDFOL explicitly combines first-order, temporal, and
 259 deontic syntax; DCEC adds agent-indexed cognitive operators and event-oriented structure. Their
 260 source modules expose these constructors, but constructor availability is distinct from complete
 261 backend coverage. [artifact S20–S21] The same interpreted norm may yield a deontic force/scope
 262 view, frame-based roles, temporal constraints, event/fluent relations, graph traversal edges, and a
 263 supported prover encoding. These are complementary task views. A predicted view distribution
 264 estimates which interfaces deserve attention; it is not a mixture of truth values. The shared symbol
 265 and provenance records permit joins, while each view retains its own semantic profile. [artifact
 266 S09,S31]

267 Bare operator letters are unsafe identifiers: F can mean prohibition or eventually, and G can denote a
 268 cognitive goal or a temporal always operator. Family, profile, operator kind, arguments, and nesting
 269 must survive serialization. In particular, $O(Gp)$ and $G(Op)$ are not equated without a profile-specific
 270 commutation result.

271 6.2 Loss-aware translation

272 A translation record must state the source/target views, profile, signature map, assumptions, supported
 273 constructs, named losses, preserved property, and checker evidence. Preserving satisfiability is
 274 different from preserving the truth of an arbitrary query; one-way abstraction is different from
 275 equivalence. Translation validation examines the produced transformation rather than trusting every
 276 invocation of a compiler [7].

277 A concrete legacy path makes the need visible: the inspected TDFOL-to-FOL converter removes
 278 unary temporal and deontic operators and replaces a binary temporal formula by conjunction. This
 279 may serve as a lossy projection, but it cannot generally carry a modal theorem. The paper does not
 280 attribute this behavior to every adapter. For example, with p false in the actual world and true in every
 281 ideal world, $O(p)$ holds while p does not. On the trace $p = [true, false]$, p holds now but Gp does
 282 not. For strong until, q true now makes $p U q$ true regardless of p now, contradicting the proposed
 283 replacement $p \wedge q$. [artifact S18]

284 Encoding a proposition as a predicate such as obligatory(p) also does not supply its semantics. The
 285 representation must distinguish quotation/reification from applying a predicate to a term; relational or
 286 higher-order embeddings must interpret the modality, not simply preserve its spelling.

287 6.3 A conditional proof-transfer rule

288 Let $\Gamma \models_{\mathcal{L}_s} \phi$ be the desired source-view consequence. A translator τ maps the premises and goal
 289 into $\mathcal{L}_t$. Under bridge assumptions B , let $\mathcal{R}_B(s, t)$ relate admitted source and target models. The
 290 required directionality is:

$$s \models \Gamma \wedge \mathcal{R}_B(s, t) \Rightarrow t \models \tau(\Gamma), \quad (7)$$

$$t \models \tau(\phi) \wedge \mathcal{R}_B(s, t) \Rightarrow s \models \phi. \quad (8)$$

291 For every admitted source model satisfying Γ and B , there must be a related target model; otherwise
 292 the relation can be vacuous. If the target checker soundly establishes $\tau(\Gamma) \models \tau(\phi)$, these conditions
 293 transfer the consequence back. A proof sketch appears in Appendix B. This is a contract to discharge
 294 per supported translation, not a soundness theorem about every current extractor.

295 Cross-source composition introduces further obligations: entity identities must align; timestamps
 296 and units must agree; domain assumptions and exception priorities must be compatible; and the joint

premise set must be consistent in the relevant sense. Under classical logic, inconsistent premises entail any conclusion. A system that silently adds an assumption to close a proof has changed the problem rather than solved it.

7 Worked example: learned guidance and cross-view reasoning

Consider a synthetic coding requirement: “Protected resources must not be written without approval. Each write must be audited within two steps.” It is a specification example, not legal advice or a production experiment. One source supplies the requirement, a second supplies implementation text, and a third supplies observations. Let a be an actor, r a resource, and t a discrete trace position.

The normative view may represent the first sentence as:

$$\text{Protected}(r) \wedge \neg \text{Approved}(a, r, t) \Rightarrow O_{a,t,\theta}(\neg \text{Write}(a, r, t)). \quad (9)$$

The policy-to-compliance bridge defines which observed behaviors count as violations. It does *not* derive actual non-writing from the obligation. The compliance queries are:

$$Q_1 = \forall a, r, t : \text{Protected}(r) \wedge \neg \text{Approved}(a, r, t) \Rightarrow \neg \text{Write}(a, r, t), \quad (10)$$

$$Q_2 = \forall a, r, t : \text{Write}(a, r, t) \Rightarrow \exists u \in [t, t + 2] : \text{Audit}(a, r, u). \quad (11)$$

A deterministic implementation model sets $\text{Write} = \neg \text{Protected} \vee \text{Approved}$. Substitution establishes Equation 10 in that model. An implementation that always writes supplies a counterexample when the resource is protected and approval is false. Transferring either result to actual source code requires a justified extraction or refinement relation; finding the word “approve” in a function name is insufficient.

For Equation 11, an observed write at step 0 and a matching audit at step 1 satisfy the bounded requirement for that event. A matching audit at step 3 violates the two-step requirement only when the capture is complete through the relevant deadline and no earlier matching audit exists. An incomplete window yields unknown, not compliance and not proven violation. The same-looking name in a different tenant or request cannot be joined without an admitted identity mapping. A global program guarantee additionally requires reasoning over all modeled executions, not one successful log.

The shared space reuses typed actor/resource/event identities, source maps, and explicit bridge assumptions. A Legal declaration can state the constraint, an Intent declaration can describe the proposed workflow, and a Security declaration can supply implementation properties. The intent is not an observed execution: explicit identity, action, time, and refinement mappings join these views. The resulting behavioral theorem, trace observation, and policy-applicability decision remain different conclusions. [artifact S01,S09]

During training, paired variants of this example can change “must” to “may”, move the exception, or change the audit window. The compiler and view metrics provide candidate targets; independent annotations determine whether the intended change survived. A family/view head may identify a temporal omission, a decompiler-plan feature may localize the missing phrase, and a proof-feedback head may rank a useful obligation. None substitutes for the actual current-profile check. This example illustrates how the representations cooperate; it is not reported as an executed learned-model experiment.

8 Evidence and evaluation plan

This revision audits source code at one frozen repository snapshot. It does not execute the full repository, train a new checkpoint, establish a live CUDA speedup, or show that every supported head is enabled in the selected deployment. Existing CUDA tests check real gradient work, CPU parity, clipping, and resident parameters, but they are skipped without suitable hardware and use convenience fixtures. Their presence is not a new passing test run. [artifact S32]

The earlier standalone reference suite remains in the supplement: all twelve finite semantic checks met their expected outcomes, including 26 disagreements between strong-until and conjunction among 64 three-step trace pairs. These are illustrations, not accuracy measurements for the repository’s compiler or autoencoder. No training gain is inferred from them.

Table 1: Measurements needed to distinguish learning from copying or teacher self-consistency.

Question	Required comparison
Generalization	Sample-memory on/off; disjoint final test set; supervised target provenance
Reconstruction	Embedding MSE/cosine versus source-withheld text and independently labeled IR facets
Multi-view learning	Per-family/view CE and coverage; modal/exception/quantifier minimal pairs
Proof feedback	Heads off/on; calibration, abstention, actual checker outcomes, protected-objective invariance
Compiler transfer	No guidance versus admitted guidance; fixed-canary and held-out fidelity; rollback
Economics	Compilation, target construction, gradient updates, validation, search, retries, and provider cost

The empirical evaluation must isolate distinct questions (Table 1). In particular, report deterministic-only, adaptive-with-memory, adaptive-without-memory, source-free promoted guidance, and proof-head ablations. Also ablate BM25/vector/graph context, deterministic versus optional premise ranking, and Hammer versus Leanstrat assistance. Use matched tasks, model budgets, teacher targets, and acceptance checks. Keep source-family and temporal test splits separate from the repeatedly consulted selection set. **[To complete: Supply frozen checkpoint/configuration identities, training and split counts, teacher-vector provenance, enabled heads, held-out fidelity, and independent proof-transfer results.]**

A controlled compiler/realizer cycle can consistently omit the same fact; a solver can prove that incomplete formalization; and a student can learn the same omission from its teacher. Final evaluation must therefore use independent source interpretations, countermodels, and useful proof coverage, not only parse success, round-trip agreement, or an aggregate score. Include abstentions and unsupported cases in the denominator and report actual extraction, annotation, learning, repair, proof, and review costs.

9 Related work, limitations, and conclusion

Mathematical autoformalization [1, 2, 9] motivates learned statement generation, but our source pipeline combines domain-specific parsing, structured targets, adaptive representation heads, and checked compilation guidance. Learned perception plus symbolic execution is established in neural-symbolic reasoning [8]; here provenance, modal scope, and separate proof-feedback objectives are material. We do not claim to originate multi-logic interoperability: institutions formalize signatures, models, and satisfaction [4], while LogiKEy combines normative logics by HOL embedding [3]. Frame logic, event calculus, and hammer reconstruction are established foundations [5, 6, 10].

The main limitations are teacher bias, possible sample-memory shortcuts, repeated validation-set selection, unmeasured checkpoint activation, dynamic source behavior, lossy legacy translations, and bounded backend coverage. The proof space is neither a universal latent space nor a universal decision procedure. A trained view classifier, valid source map, or high reconstruction score does not by itself establish faithful interpretation. Result kinds remain distinct: theorem proof, satisfiability, finite runtime observation, evidence readiness, and policy approval. [artifact S19]

The methodology is best understood as a coupled learning and compilation system: symbolic structure supplies auditable targets; trainable shared representations expose systematic errors; constrained proposals and fixed-canary admission improve the compiler-facing path; and modality-aware views support explicitly justified reasoning. The next result to establish is whether this cycle improves independent source fidelity and useful cross-view proof coverage on genuinely unseen material, at a favorable total cost.

References

- [1] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35*, 2022. <https://arxiv.org/abs/2205.12615>.
- [2] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. arXiv:2302.12433, 2023.
- [3] Christoph Benzmüller, Xavier Parent, and Leendert van der Torre. Designing normative theories for ethical and legal reasoning: LogiKEy framework, methodology, and tool support. *Artificial Intelligence*, 287:103348, 2020. doi:10.1016/j.artint.2020.103348.
- [4] Joseph A. Goguen and Rod M. Burstall. Institutions: Abstract model theory for specification and programming. *Journal of the ACM*, 39(1):95–146, 1992. doi:10.1145/147508.147524.
- [5] Michael Kifer, Georg Lausen, and James Wu. Logical foundations of object-oriented and frame-based languages. *Journal of the ACM*, 42(4):741–843, 1995. doi:10.1145/210332.210335.
- [6] Robert A. Kowalski and Marek J. Sergot. A logic-based calculus of events. *New Generation Computing*, 4:67–95, 1986. doi:10.1007/BF03037383.
- [7] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems*, LNCS 1384, pages 151–166, 1998. doi:10.1007/BFb0054170.
- [8] Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Joshua B. Tenenbaum. Neural-symbolic VQA: Disentangling reasoning from vision and language understanding. In *Advances in Neural Information Processing Systems 31*, 2018. <https://arxiv.org/abs/1810.02338>.
- [9] Albert Q. Jiang, Wenda Li, and Mateja Jamnik. Multi-language diversity benefits autoformalization. In *Advances in Neural Information Processing Systems 37*, 2024.
- [10] Albert Qiaochu Jiang, Wenda Li, Szymon Tworowski, Konrad Czechowski, Tomasz Odrzygóźdź, Piotr Miłoś, Yuhuai Wu, and Mateja Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers. In *Advances in Neural Information Processing Systems 35*, 2022.

405 A Anonymous implementation map and claim status

406 The identifiers below are source-inspection references, not newly measured experiments. The private
 407 author ledger maps each to the exact repository revision, path, inspected range, and available blob
 408 identity. A submission supplement containing the frozen implementation still needs to be prepared
 409 and anonymized; the small reference-semantics suite included with this draft is not a substitute for
 410 it. Existing source modules, documented contracts, proposed semantic conditions, and measured
 411 performance are different evidence classes.

Table 2: Module roles inspected for this paper.

ID	Source role	Supported claim and boundary
S01	IR-family identity architecture	Domain-neutral canonicalization, provenance, and separate domain/result roles; not shared ontology.
S02	Generic compiler contracts	Configuration, artifacts, obligations, and grounded unsupported diagnostics; protocol module is not an automatic extractor.
S03	Source-grounded sample contracts	Immutable normalized input; source/span/assumption checks, not domain truth checking.
S04	Formalization view registry	Typed formula/registry/symbol/link records; declared cross-view labels are not proofs.
S05	Typed syntax core	Many-sorted FOL nodes plus versioned extensions, elaboration and signature checks.
S06	Version-three artifacts / domain slices	Source/expression lineage before backend requests; dual-read legacy adaptation.
S07	Canonical compiler/round-trip architecture	Source-withheld reconstruction, pinned policy, bounded rule shape, distinct identity profiles.
S08	Canonical structured-text compiler	Caller vocabulary and deterministic rule projection; error, abstention, disclosed partial output.
S09	Legal multi-view adapter	Separate source declaration and shared projection identities; registered force/role/time/event views.
S10	Legal modal parser	Deterministic cue and scope scaffolding with domain-specific editorial handling.
S11	Modal registry	Named families and ambiguity profiles; registry presence is not solver qualification.
S12	Text-to-FOL converter	Predicate/quantifier/operator extraction, optional NLP and fallback, syntax validation.
S13	Python security extractor	AST plus keyword/docstring seed policies and events, explicitly heuristic.
S14	Security-model implementation guide	Bounded finite-model claims, temporal monitors, and source projections; not universal code security.
S15	General multimodal processor	Text/metadata/embedding interface; random fallback embeddings are not symbolic evidence.
S16	Image-processing search excerpt	Placeholder in the inspected general image path; specialized alternative routes not audited.
S17	Autoencoder formalization advisor	Unverified view/premise/compression advice and bounded repairs; no new learned performance claimed.
S18	Legacy temporal/deontic converters	Concrete modal erasure and approximate lowering; this path cannot justify unrestricted proof transfer.
S19	External-prover architecture	Solver-local outcomes versus native reconstruction, monitors, readiness and policy.

ID	Source role	Supported claim and boundary
S20	TDFOL syntax core	First-order, deontic, and temporal constructors with explicit term sorts; no universal embedding inferred.
S21	DCEC native syntax core	Agent-indexed belief, knowledge, intention and normative constructors; backend/theory support remains a separate claim.
S22	Legal sample builder	Source text, modal IR, frame and embedding targets; default convenience embedding is explicitly mock.
S23	Deterministic modal codec	Local parsing, spaCy feature hashing, BM25 frames, decompilation and consistency inputs.
S24	Adaptive modal autoencoder	Shared parameter state; vector decode; generalizable transactional training; separate proof heads; guidance and sample-memory modes.
S25	Cost-gated modal loop	Codec, graph import, prover signals, optional model repair, and introspection rollout.
S26	Packed tensor boundary	Stable feature-row gathering, parameter residency, optional CUDA, CPU reference and explicit fallback.
S27	Packed loss implementation	Masked family/view CE, MSE, cosine, norm guard and parameter regularization.
S28	Packed optimizer step	FP32 loss reduction, optional BF16 activations, accumulation, backward, clipping, SGD-style update and changed-row scatter.
S29	Local optimization usage guide	Loss-targeted TODO seeding, parameter updates and re-evaluation; guide is not run evidence.
S30	Learned-guidance promotion	Source-free stable export, canonical contract resolution, paired canary guards and rollback.
S31	Multi-view target builder	Adapter-derived losses and configured view-distribution targets; target generation is not a universal semantic oracle.
S32	Packed training tests	Tests for nonzero gradients, CPU parity and residency; skipped when hardware absent and not executed in this audit.
S33	Constrained IR decoder	Typed productions, fixed vocabulary and valid-token/grammar/binder/type/family masks; not a claim of trained token-generation quality.
S34	GraphRAG thin-client implementation	Sharded lexical scores, CID adjacency and bounded walks; the declared vector CLI mode is not implemented in this route.
S35	FAISS vector store	Actual index/metadata/identifier interface with dependency checks; no claim about a trained encoder or measured retrieval gain.
S36	Optional Hammer selector guide	Deterministic overlap baseline and pinned graph-based scoring; supplied weights are hand-authored, not a trained checkpoint.
S37	ITP Hammer user guide	Bounded supported-fragment translation, opt-in solver portfolio, native reconstruction and result states; documented contract, not a new live run.
S38	Logic Tactician planner	Pure, deterministic source ordering and acyclic subgoals; optional pinned reorder/nomination; no proof execution in planning.
S39	Leanstrat proof-assistant lane	Advisory proposals plus Lean checking and anti-bypass policy; model configuration does not establish execution or success.

ID	Source role	Supported claim and boundary
S40	Legal Hammer/Leanstrat rollout run-book	Separate draft, trusted feedback, stable export and patch artifacts; fixed stages, canary gates and rollback evidence.

412 **Integration status to fill.** For each claimed source-to-proof route, publish the exact frontend,
413 domain adapter, syntax/elaboration version, logical profile, backend, optional features actually
414 enabled, and result checker. Report an unsupported or unavailable route honestly. An importable
415 adapter, passing schema test, old benchmark guide, or advertised feature list is not current end-to-end
416 evidence.

417 B Preservation contract and proof-transfer argument

418 Let $\mathcal{S}_B$ be the source models admitted under explicit bridge assumptions B . Let τ be fixed on a
419 specified premise and goal language. The contract is not required to cover every source sentence.

420 **Proposition 1** (Conditional semantic consequence transfer). *Suppose for every $s \in \mathcal{S}_B$ satisfying*
421 *Γ , there exists a target model t such that $\mathcal{R}_B(s, t)$ holds. Suppose related models preserve premises*
422 *in the direction of Equation 7 and reflect the goal in the direction of Equation 8. If a sound target*
423 *checker establishes $\tau(\Gamma) \models \tau(\phi)$, then $\Gamma \models_{\mathcal{S}_B} \phi$.*

424 *Proof.* Take any $s \in \mathcal{S}_B$ satisfying Γ . Existence supplies a related t . Premise preservation gives
425 $t \models \tau(\Gamma)$. Target consequence gives $t \models \tau(\phi)$. Goal reflection gives $s \models \phi$. The argument holds for
426 each such s . $\square$

427 This is a paper-level conditional argument, not a machine-checked theorem of the repository. It
428 does not establish that the source extraction was faithful, that B was appropriate, that the model
429 class was nonempty, or that a legacy converter satisfies the two preservation directions. Test consis-
430 tency/reachability separately to avoid reporting a vacuous result as practical assurance.

431 **Suggested translation receipt.** The following is specification notation rather than a newly imple-
432 mented replacement for existing contracts:

```

433     source-view, target-view, source-formula-root,
434     target-formula-root,
435     signature-map, interpretation-profile, bridge-assumptions,
436     preserved-property, preservation-direction,
437     supported-fragment,
438     named-losses, source-map-root, checker-environment,
439     checker-result, proof-or-countermodel-reference, status

```

440 A consuming query must check that the receipt’s preserved property and direction are adequate for that
441 query. Equi-satisfiability alone cannot establish a mapping of every model’s temporal observations.
442 A loss flag cannot be ignored because a later solver returned UNSAT.

443 **Multi-source joins.** For a document actor, code parameter, and event field, include a reviewed
444 alignment claim rather than collapsing all matching strings. Bind namespace, tenant, instance identity,
445 role, validity interval, source authority, and resolution method. Time bridges must specify clock origin,
446 monotonicity, calendar/duration semantics, timezone where relevant, and unit conversion. Missing
447 evidence remains unresolved. The result should retain all bridge assumptions so that revocation or
448 source changes can invalidate dependent conclusions.

449 **Contradictory sources.** Classical theorem provers may derive any goal from inconsistent premises.
450 Do not repair inconsistency by suppressing an inconvenient observation or promoting a model’s
451 confidence. Partition contexts, identify conflicting premises, request authoritative resolution, or
452 choose a separately justified inconsistency-tolerant semantics. Such a change is a logic-profile change
453 and must not be hidden inside a translation adapter.

Table 3: Minimal distinctions to preserve across source, IR, and backend.

Distinction	Counterexample or ambiguity	Correct boundary
Obligation vs fact	A required event has not happened. Ideal worlds satisfy p while the actual world does not.	Do not infer p from $O(p)$.
Permission vs occurrence	An action is allowed but never taken.	Permission is not an observation.
Prohibition vs negation	A forbidden action can occur and violate the norm.	Keep normative force distinct from actual falsity.
Knowledge vs belief	An agent believes an incorrect claim.	Factivity requires the knowledge profile and appropriate frame condition.
Always vs now	p now, not p at a later step.	Preserve the trace interval, not just p .
Until vs conjunction	q holds now while p does not. Strong until holds, conjunction does not.	Preserve temporal operator semantics.
Quantifier order	Each user has some key, but no key belongs to every user.	Preserve binder order and identity.
Only if vs if	“Write only if approved” differs from “if approved then write.”	Check implication direction independently.
Exception vs conjunction	“Must audit unless exempt” is not unconditional auditing or an audit of only exempt cases.	Preserve exception scope and priority.
Observed absence vs negation	A partial log has no audit event.	Unknown unless capture/deadline completeness is established.
Same spelling vs identity	A log’s “admin” and a policy’s “admin” belong to different tenants.	Reject an ungrounded entity join.
Round-trip vs fidelity	Both compiler and realizer drop the same exception.	Add independent semantic annotation and negative cases.

455 The core registry names additional profiles, including dynamic, conditional normative, frame, and
 456 hybrid views. Before evaluating such a view, state its actual interpretation. An action modality may
 457 use a transition relation; an event-calculus fluent uses time and event axioms; a frame-based ontology
 458 may use inheritance and method relations. A cognitive operator may be hyperintensional rather than
 459 an ordinary Kripke box. These are design choices to bind, not silently infer from an operator name.

460 **D Executable reference semantics**

461 The included `evaluation/reference_semantics.py` uses only the Python standard library. It
 462 checks the paper’s finite examples, not the project modules. The expected-result file is generated by
 463 the same script and is therefore a reproducibility artifact, not an independent assurance certificate.
 464 No timed performance or learned accuracy claim is derived from it.

```
465     python evaluation/reference_semantics.py
466         -output evaluation/reference_results.json
```

467 For the illustrative normal deontic check only, O is a universal modality and P its existential dual,
 468 $Pp = \neg O\neg p$; seriality supports the tested implication. This does not equate strong explicit permission
 469 with weak dual permission in every normative logic. The suite enumerates all 16 relations on two
 470 worlds and the relevant valuations for normal modal distribution; reflexive relations for factivity;
 471 serial relations for the D implication; all length-three Boolean traces for strong until; and finite
 472 request/trace relations for the worked example. Additional explicit witnesses distinguish obligations

473 from facts, nonfactive belief, present truth from always, quantifier order, incomplete trace capture,
474 unaligned identities, and lossy round trips.

Table 4: All twelve reference checks. “Expected” includes finding a counterexample when the tested shortcut is invalid.

Check	Cases	Expected outcome
Normal modal distribution	512	No finite countermodel
Reflexive factivity	32	No finite countermodel
Serial <i>D</i>	72	No finite countermodel
Obligation not actuality	1	Explicit separating model
Belief without factivity	1	Explicit separating model
Until not conjunction	64	26 disagreements
Always not current truth	1	Explicit separating trace
Quantifier order	1	Diagonal-relation witness
Guarded writes	4	Safe relation 0; unsafe 1 violation
Complete audit window	16	1 violating assignment
Incomplete/unaligned traces	3	Unknown, unresolved, bounded satisfaction
Round-trip shared loss	1	Equal recompiled IR; unequal source meaning

475 A finite absence of counterexamples is not a proof of unbounded correctness. The standard semantic
476 laws have their own mathematical proofs; these executable cases merely ensure the illustrative
477 definitions behave as described.

478 E Pipeline manifest, evaluation protocol, and result fields

479 **One tested combination.** Freeze the exact repository commit, loaded module paths, dirty overlays if
480 permitted, domain adapters, schema versions, source artifacts, vocabularies, review profiles, modality
481 profiles, compiler configuration, selected models/tokenizers, and solver/native checker versions. Do
482 not combine one branch’s compiler with a different branch’s results and call it one tested system.
483 This draft does not inspect unpushed local worktrees.

484 **Three evaluation populations.** Use: (i) controlled minimal pairs with independent expected
485 meanings; (ii) held-out natural documents, code, and traces with source-family/time splits and
486 reviewed entity mappings; and (iii) extracted-media examples only after the concrete extraction route
487 is installed and measured. Separate generator-authored examples from real held-out material. Keep
488 alternative correct formalizations and abstentions instead of forcing a unique string match.

489 **Annotations and leakage.** Freeze the intended propositions, logical profile, exceptions, temporal
490 intervals, ambiguous readings, source-map spans, and admissible assumptions before evaluating
491 model output. Annotators and proof scorers should not use candidate generation as their only oracle.
492 Source derivatives, paraphrases, translations, views, and adjacent sections can leak across splits.
493 Source-withheld realization must block original text, full source maps, file/path recovery, and access
494 to gold IR; it must not quietly retrieve the withheld source.

Table 5: Methods to compare under matched tasks and reasoning budgets.

Arm	Method
A	Direct model-to-target formalization with the same target checker.
B	Deterministic bounded compiler, declared vocabulary, and explicit abstention.
C	Typed multi-view representation with source grounding, but no admitted cross-view proof transfer.
D	C plus checked property-specific bridges and compatible cross-source premise assembly.
E	D plus bounded learned view/premise advice, with frozen checkpoint and unchanged checker.

An unimplemented arm is unavailable, not a zero-score success. A method may be compared only on the fragments for which its semantics is specified; also report the full-population coverage/abstention rate so narrowing coverage cannot masquerade as accuracy.

Table 6: Final empirical results remain unmeasured in this draft. Fill from retained runs, not targets.

Arm	Sources / covered	Fidelity / uncertainty	Correct transfers	Cost / latency
A	[TBD]	[TBD]	[TBD]	[TBD]
B	[TBD]	[TBD]	[TBD]	[TBD]
C	[TBD]	[TBD]	[TBD]	[TBD]
D	[TBD]	[TBD]	[TBD]	[TBD]
E	[TBD]	[TBD]	[TBD]	[TBD]

Report parse success, typed elaboration, source-facet fidelity, modal/quantifier scope, source-map validity, reconstruction fidelity, consistency, supported query coverage, correct proof transfer, false transfer, countermodel correctness, abstention, and checker class separately. Use paired differences and confidence intervals; a zero error count over a small corpus does not establish universal soundness. Include setup, vocabulary curation, annotation, extraction, model, proof search, independent checking, retries, and human review costs. A deterministically emitted formula does not have zero total engineering or processing cost.

F Source-grounded query procedure

This is a composed method specification, not a claim that one existing function already performs every step.

1. Bind a query to its source references, target property, ontology, allowed assumptions, and acceptance profile.
2. Extract and normalize candidate declarations; retain unresolved observations and possible interpretations.
3. Elaborate each formula against its signature; preserve modality, scope, time, exception, and source lineage.
4. Resolve cross-source actors, events, resources, and units through admitted bridge records.
5. Select only a target view that supports the required constructs. Unsupported loss causes abstention or explicit partial analysis.
6. Check compatibility and nonvacuity of the selected premises; create property-preservation obligations for every relied-upon translation.
7. Invoke an available solver or prover under exact theory, environment, and resource bounds. Keep candidate, satisfiability, monitor, and kernel-checked outcomes distinct.
8. Reconstruct/check evidence at the assurance level required for the query. Bind the actual verified statement to the outer artifact fields.
9. Return the conclusion with all assumptions, sources, bridge records, limitations, and result kind. Unknown does not become true or false by default.
10. On counterexamples, distinguish a bad source interpretation, an inadequate abstraction, an invalid translation, and a real behavior violation before requesting a repair.

G Completion, LLM disclosure, and artifact separation

Scope of this manuscript revision. The research pass used read-only repository inspection and primary-literature research. It executed only the standalone reference-semantics script and LaTeX build/quality checks. It did not train models, run OCR/ASR, execute the repository’s full tests, run native provers, or validate a universal multimodal translation. Prior supervisor paper measurements are intentionally not reused as autoformalization results.

Disclosure to finalize. LLM assistance was used to research and draft this manuscript and its reference-example code. The author must verify the text, formulas, examples, implementation claims, and bibliography before submission. Record the actual models, dates, and material implementation/formalization/experiment uses when the final runs exist; do not infer a model version or an author review that has not occurred. The reference suite is not a claim of human-independent discovery or audited proof.

Remaining source and format work. Replace planned-result fields after freezing the source and experimental manifests. Validate actual modal bridges, source-fidelity annotations, positive proof paths, counterexamples, and availability. Prepare the anonymous implementation supplement; keep the private audit with public repository identities outside the submission. Replace the missing official checklist file rather than inventing its questions. The workshop style must remain unchanged; recheck the main-text page count after adding results. All appendices are part of the same submission PDF.

H Learning-system anatomy and terminology

The original methods draft emphasized generic learned advice. This revision examines the concrete modal autoencoder, packed tensor training, source-free feature promotion, and multi-view targets. All statements here describe the inspected implementation; no new model was trained in this manuscript preparation.

Table 7: Objects that must not all be called the same latent representation.

Object	Construction and use	What the current evidence does not establish
Canonical modal IR	Source-linked formulas, operators, predicates, roles, conditions, exceptions and frame structure.	Unsupervised invention of the schema; arbitrary whole-language correctness.
Structured round-trip IR	Seven-field rule records under a specific compiler, vocabulary and realization profile.	Full fidelity for every richer modal declaration.
Feature encoding	Token/cue, semantic-slot, frame, contract, scope, provenance and diagnostic features.	A pure text-only input when an existing parse or bridge target is used.
Decoded embedding	Numerical prediction compared against the sample embedding using MSE and cosine.	Generated natural-language reconstruction or a guaranteed information bottleneck.
Family/view distribution	Class scores over configured modal families and compiler/prover view components.	Probability that a statement is legally true or a theorem is proved.
Source-withheld text	Deterministic realization from admitted IR without originating surface access.	Proof of original-source fidelity; compiler and decoder can share omissions.
Proof-feedback heads	Bounded supervised predictions from admitted, version-bound verifier feedback.	A substitute for the next actual proof check.
Stable guidance export	Selected shared features mapped to canonical view contracts after canary admission.	Raw parameter state becoming a new semantic authority.

A useful classification. The concrete adaptive component is a feature-based, multi-head reconstruction/classification learner with sparse state and a packed tensor execution path. It should not be described as a variational autoencoder, generative adversarial network, Transformer sequence-to-sequence formalizer, or trained universal semantic bottleneck without identifying a separate implemented model and checkpoint. Other experimental mechanisms may exist elsewhere; this classification applies to the exact paths audited here.

556 **The three feedback loops.** The *representation loop* changes shared weights. The *program-*
557 *improvement loop* changes a compiler/decompiler or its admitted guidance under tests and source-
558 fidelity checks. The *proof-assistance loop* learns which obligation, premise, or route may help from
559 trusted verifier feedback. They exchange diagnostics and typed artifacts, but their parameters, targets
560 and acceptance criteria are not identical.

561 I Dataset construction and supervision provenance

562 A legal training example records citation, raw text, normalized text, embedding producer, vector,
563 parsed modal document, frame candidates/selection, parser trace and per-example loss records. The
564 builder normalizes text, obtains a deterministic parse, and ranks frames. Its default vector is a seeded
565 mock suitable for fixtures; the local codec supplies a different, spaCy-feature-hashing route, while a
566 caller may supply another explicit producer. Every dataset release should preserve the distinction
567 rather than silently labeling all vectors pretrained semantic embeddings.

568 The initial IR is therefore often *teacher-generated supervision*. Family proportions summarize opera-
569 tors already in the parse. Multi-view distributions are generated by registered adapters and configured
570 contract/routing signals, including explicit floors, caps and source-cue rules in the inspected builder. A
571 low loss against those targets demonstrates learning those associations; source truth requires separate
572 annotation. A failed syntax or unavailable prover result must retain its status instead of receiving an
573 invented successful training label.

Table 8: Recommended experiment manifest fields; values remain to be filled from an actual training run.

Field	Required value
Source root	Exact corpus release, licensing/review state, document and derivative grouping
Teacher identities	Parser, vocabulary, frame index, embedding producer, view registry and target-builder versions
Model state	Shared parameter root, sample-memory mode, enabled groups, scales and capacity limits
Splits	Training; repeatedly used selection/canary set; untouched source-family/time test set
Execution	CPU/CUDA path, library and device versions, accumulation, clipping and precision
Selection	Learning-rate candidates, objective weights, hard-example policy, timeout and deadband mode
Proof feedback	Verifier, compiler/profile fingerprint, admissible statuses, excluded partitions and duplicates
Promotion	Paired canary identity, admitted stable export, active consumer, rollback and held-out results

574 **Leakage and shortcuts.** Disable sample-indexed reconstruction memory for the main generaliza-
575 tion result. Do not pass a full gold IR into a student claimed to predict that IR from source alone.
576 Features obtainable only after an expensive complete compilation must be charged in the deployment
577 cost. Teacher evaluation on familiar targets and repeated selection on a validation set cannot substitute
578 for a final held-out test. Keep paraphrases, sections, views, synthetic variants and source-derived
579 artifacts grouped according to the declared split policy.

580 J Exact losses, updates, and validation control

581 J.1 The packed differentiable step

582 For n samples and embedding dimension d , the inspected reconstruction loss is

$$L_{\text{rec}} = \frac{1}{n} \sum_i \frac{1}{d} \|\hat{e}_i - e_i\|_2^2. \quad (12)$$

583 For a categorical family or view target, normalize the nonnegative target row and apply

$$L_{CE} = -\frac{1}{n} \sum_{i \in \mathcal{M}} \sum_j q_{ij} \log \text{softmax}(\ell_i)_j, \quad (13)$$

584 where $\mathcal{M}$ selects available target rows. The denominator in the inspected path is the total sample count,
 585 not necessarily the count of active rows. The cosine term is evaluated on nonzero target/prediction
 586 norms. The broad norm-ratio guard is zero within its safe region and penalizes extreme ratios. L2
 587 regularization is normalized by the number of parameters. Not every term is active for every selected
 588 head.

589 Microbatches contribute their globally normalized losses before gradients are accumulated. The
 590 implementation calls backward, rejects nonfinite packed loss metrics, computes and clips the gradient
 591 norm, and performs

$$\Theta \leftarrow \Theta - \eta \text{Clip}(\nabla_{\Theta} L), \quad (14)$$

592 with a bounded step. The inspected code directly adds the scaled negative gradient; it does not
 593 implement Adam moments. Changed parameter rows are scattered back into the canonical transaction-
 594 aware maps. Device residency is an execution optimization, not a different symbolic semantics. Some
 595 configured groups may have no active parameters; a reported forward-only no-op must not be counted
 596 as learning progress.

597 J.2 The outer model-selection step

598 The generalizable trainer evaluates family, reconstructed-embedding, global-view, detailed-view,
 599 and combined update candidates. It tries learning-rate multipliers, may select hard examples, can
 600 cap target construction and pre-screen candidates, and compares candidate objective changes with
 601 per-metric regression limits. It temporarily applies candidate state, evaluates, and rolls it back before
 602 deciding which candidate to commit. The evaluated objective also includes view/structural losses,
 603 which need not be differentiable tensors.

604 This is an inner numerical update surrounded by guarded discrete selection. Calling every stage
 605 backpropagation would be wrong. Equally, describing the tensor path as only heuristic tweaking
 606 would miss its actual backward and parameter-update operations. If validation samples are absent,
 607 evaluation falls back to training examples; that case must not be labeled held-out validation. A
 608 nonzero deadband changes the acceptance policy and must be disclosed, not buried as a numerical
 609 implementation detail.

610 J.3 Proof-feedback heads and objective isolation

611 The categorical heads are obligation family, trusted outcome, and reconstruction success. The
 612 multilabel heads are semantic slots, premise family, proof-route availability, and minimal failing
 613 contract. The default vocabulary and family-context sizes are bounded. A head can abstain when its
 614 confidence or available labels are inadequate.

615 Training accepts only records passing trust, partition, and compiler/toolchain version checks. Duplica-
 616 te IDs are not trained repeatedly. The first admissible version may pin previously empty proof-head
 617 state; mismatches are excluded. The implementation compares protected objective fingerprints and
 618 reports a proof-loss weight of zero in the primary representation objective. This is a useful causal
 619 design: proof feedback can teach prediction and route selection without being permitted to rewrite
 620 the statement solely to maximize provability. Actual downstream benefit still needs an ablation.

621 K Constrained decoding and learned-to-symbolic promotion

622 K.1 Grammar controls the candidate language

623 The decoder implementation declares typed productions and a frozen vocabulary of 143 tokens,
 624 including special, binder, operator, type, family, proof, tactic, source, identifier and production
 625 categories. Its declared masks are valid token, grammar, binder, type and family; its search bounds
 626 are 16 beam entries and 64 decode steps. These are implementation constants, not results of a newly
 627 trained model. The exact profile and any changes belong in the run manifest.

This provides an engineered output language in which a model may rank candidates. It prevents impossible JSON shapes or invalid family/type combinations from receiving metric credit. It does not guarantee correct grounding, truth, faithful negation or exception scope. A fallback involving an admitted reference or gold artifact is not an independent blind prediction and must be excluded or reported separately in that evaluation condition.

K.2 Feature export is not source copying

The stable learned export restricts outputs to reusable feature groups such as compiler contracts, cycle consistency, decompiler surface templates, logic-view contracts, repair lanes and semantic slots. The inspected promotion code explicitly rejects raw-source, source-span, sample-memory, and token-copying markers. It resolves the export against registered contracts and checks fixed-sample paired metrics. Missing guardrail evidence, source-copy regression, symbolic-validity regression or an unfixed sample set blocks guidance emission.

The product is a content-addressed guidance record: target contract and family, bounded features, repair suggestions, canary evidence, source-export identity and rollback metadata. Admission of this record is not itself a proof that a specific consumer enabled it. For an end-to-end claim, record the loaded guidance identity and show that the actual compiler or supervisor consumed it under the evaluated configuration. A branch containing training code and a canonical compiler configured with learning disabled are not one measured learned pipeline.

K.3 Program synthesis changes the algorithm, not just the weights

Introspection compares predicted and target families/views, decoded and target embeddings, semantic-slot diagnostics, source reconstruction, and feature-group ablations. The TODO mechanism distinguishes learned-parameter actions from deterministic program-synthesis actions. For example, loss caused by poor shared family ranking can lead to a parameter update; loss caused by an unhandled exception pattern can lead to a parser/realizer patch proposal. Such a patch must be checked against preserved source facets and regression cases before its behavior becomes canonical.

The local modal TODO supervisor and the broader external agent supervisor are not assumed identical. The paper’s artifact must identify the actual entry point, generated task/proposal, implementation executor, validation record, and activation path used in the experiment. No new autonomous execution is launched by this manuscript.

L Using IRs together: a concrete explanatory trace

Consider the synthetic sentence: “A permit holder must file a notice within ten days after approval, unless exempt.” A bounded canonical rule may preserve modality O, actor permit holder, action file, object notice, an approval condition, an exemption, and a ten-day temporal qualifier. This is an explanatory example, not a newly run parser result.

Table 9: Complementary views and what must be joined before a proof.

View	Contribution	Required bridge/evidence
Deontic	An obligation with condition and exception scope.	Reviewed normative profile; exemption precedence.
Frame/F-logic	Holder, permit, notice, and approval roles and relations.	Grounded identity and role mapping, not string equality.
Temporal/TDFOL	Approval anchor, permitted clock/unit interpretation, deadline relation.	Correct interval and time semantics; no modal erasure.
Event calculus	Approval and filing events; resulting fluent transitions.	Admitted event identity, timestamp and transition axioms.
Graph projection	Retrieval, links, dependency and provenance traversal.	Graph edge is evidence or structure, not automatically a theorem.
Prover encoding	A stated compliance query under explicit premises and theory.	Valid translation and independently checked result.

View	Contribution	Required bridge/evidence
Decompiler	Inspectable reconstruction of force, roles, conditions, exception and time.	Withheld source; independent original-fidelity check.

The statistical heads may identify that the example needs deontic, temporal and event views; a frame model can nominate a role alignment; the decompiler can expose a dropped exception; and proof feedback can suggest a relevant premise or route. The symbolic pipeline must still resolve those suggestions to exact declarations and checked translations. A nearby embedding does not merge two agents or prove that the obligation was fulfilled.

Cross-domain Legal, Security and Intent declarations can share source and symbol references and compose a query through reviewed bridge assumptions. Their canonical domain identities remain different. A shared neural feature backbone or transferable parameter group is a statistical hypothesis to test; it is not a license to reinterpret one domain’s result as another domain’s authorization or proof.

M Training experiments and paper-completion fields

Table 10: Ablations for the actual learning method. All outcomes are pending a frozen run.

Arm	Condition	Main question
T0	Deterministic codec and view adapters	Teacher baseline, coverage and original fidelity
T1	Adaptive learner with sample memory	How much apparent gain is memorization?
T2	Same learner, shared parameters only	Does prediction generalize to unseen source families?
T3	T2 plus isolated proof heads	Does trusted feedback improve routing/calibration without harming fidelity?
T4	T3 plus admitted stable guidance	Does the actual compiler/realizer improve when guidance is consumed?
T5	Bounded compiler/decompiler code improvements	Does algorithm repair add value beyond numerical updates?

Report active model state and input features for every arm. If a target or feature is inaccessible at real inference time, its cost or oracle status must be visible. The final test set must not select line-search steps, hyperparameters, canary thresholds, grammar rules or compiler patches. An improved embedding metric is not a substitute for source-facet accuracy. Include modal-force flips, exception/condition swaps, quantifier scope, actor/recipient exchange, changed time anchors and duplicate-source leakage probes.

Table 11: Results to fill from retained training and inference records, not from code defaults.

Arm	Source/split counts	Held-out fidelity	Proof/route benefit	Total cost
T0	[TBD]	[TBD]	[TBD]	[TBD]
T1	[TBD]	[TBD]	[TBD]	[TBD]
T2	[TBD]	[TBD]	[TBD]	[TBD]
T3	[TBD]	[TBD]	[TBD]	[TBD]
T4	[TBD]	[TBD]	[TBD]	[TBD]
T5	[TBD]	[TBD]	[TBD]	[TBD]

No percentage gain, active deployment state, training corpus size, or completed qualification is inferred from the presence of these modules. Conversely, a historical no-go report for one training campaign does not prove that every newer learner is untrained or unavailable. A named checkpoint, exact configuration, target provenance, activation receipt and independent evaluation are the required evidence.

683 N Retrieval substrates and premise formation

684 The architecture has more than one retrieval implementation. This appendix separates lexical lookup,
685 embedding-neighbor search, graph navigation, and formal premise admission. It does not claim that
686 a single request automatically combines every index or that a learned fusion policy has been trained.
687 The relevant inspected sources are the legal sample builder, GraphRAG thin client, FAISS store, and
688 Hammer selector contract. [artifact S22,S34–S36]

689 N.1 BM25 in sample construction and corpus lookup

690 The sample builder invokes a BM25 frame selector to produce candidates and a selected ontology
691 frame. This is supervision provenance: a selected frame may be useful but is not automatically
692 a human-validated semantic label. The GraphRAG thin client is a different consumer of lexical
693 indexing. It tokenizes the query, locates relevant posting shards, and computes a term contribution of
694 the form

$$s(t, d) = \text{idf}(t) \frac{\text{tf}(t, d)(k_1 + 1)}{\text{tf}(t, d) + k_1(1 - b + b|d|/|\bar{d}|)}. \quad (15)$$

695 For one inspected posting schema, term frequency combines configured title and body weights.
696 However, a separate schema branch keyed by legal identifiers sums the supplied term frequencies
697 instead. Consequently, a response labeled “bm25” is not by itself evidence that every data schema
698 used the same full scoring formula. The manifest, posting schema, scoring branch, tokenizer, and
699 parameters belong in a reproducible run record. [artifact S34]

700 Lexical retrieval is useful for citations, names, identifiers, and distinctive vocabulary. It can also
701 miss paraphrases, rank a superficially matching exception above a controlling rule, or omit a relevant
702 provision. Its score supplies ordering, not entailment, completeness, or applicability. Frame-selection
703 errors can propagate into compiler-derived supervision unless independent checks expose them.

704 N.2 Vector indexing is separate from representation learning

705 The FAISS store maintains numerical indexes, per-collection metadata, and mappings between
706 external identifiers and internal index positions. Its inspected constructor requires real FAISS and
707 NumPy availability; legacy mock definitions in the file are not evidence of a successful real store
708 query. An evaluated route must identify the actual backend and status. [artifact S35]

709 Three operations must be recorded separately: producing a vector with a frozen or trained encoder;
710 fitting any index structures that a selected index type requires; and changing semantic model param-
711 eters. The adaptive autoencoder’s embedding reconstruction is a fourth, separately specified numerical
712 operation. A published numerical vector does not prove that the corresponding source was formally
713 interpreted, and a store over vectors does not establish that those vectors came from this autoencoder.

714 The thin client declares a vector subcommand and contains an embedding helper, but its inspected
715 dispatch does not execute a vector query. Actual vector search must therefore be attributed to another
716 verified entry point rather than to that declaration. This is a route-specific gap, not a claim that the
717 repository lacks vector indexing. [artifact S34,S35]

718 N.3 Graphs connect evidence without silently adding axioms

719 The thin client can retrieve incoming/outgoing adjacency records and execute bounded walks over
720 CID-linked nodes. Its results retain neighbor identifiers and edge metadata. A graph can connect
721 a source passage to a definition, entity, related artifact, or dependency; an index edge does not
722 necessarily assert a logical implication. [artifact S34]

723 At the query boundary, lexical and vector hits can nominate seeds, graph navigation can expand their
724 context, and exact artifact identities can retrieve declarations. This is a proposed composition of the
725 available interfaces, not a measured fusion algorithm. A premise-admission stage must then inspect
726 source lineage, entity and temporal alignment, domain applicability, declared authority, and the
727 relevant interpretation. Keep “retrieved”, “admitted premise”, “assumed”, and “proved” as separate
728 states.

729 **Identity and reuse.** The inward IR kernel binds canonicalization profile, domain, schema, collection
730 semantics, and payload into content identity. A CID identifies those bytes; it is not a proof of meaning.
731 Reuse of a proof or translation must also bind the relevant premises, theory, toolchain, policy, and
732 checker evidence. Similarity retrieval is not an exact-result cache, and equality of an embedding is
733 not equality of canonical declarations. [artifact S01]

734 O Tactician, Hammer, and Leanstral: distinct computational roles

Table 12: Planning, proposal, and checking do not have interchangeable authority.

Component	Computation or output	Acceptance boundary
Tactician	Policy-bound route ordering and finite acyclic subgoals; stop/abstain reasons.	Pure plan; no theorem or execution result is produced.
Hammer selector	Rank existing premise candidates from reproducible features.	Relevance score is not logical implication; sources still require admission.
ATP portfolio	Search translated first-order problems using allowed Vampire/E routes.	Supported translation, explicit execution permission, native reconstruction.
SMT portfolio	Search admitted SMT-LIB encodings using allowed Z3/cvc5 routes.	Solver-local outcome retains theory and scope; not automatically an ITP theorem.
ITP frontend and reconstructor	Capture the native goal and reconstruct a native tactic/term candidate.	The originating Lean/Coq/Isabelle checker accepts the proof in a pinned environment.
Leanstral	Suggest proof edits, strategies, or bounded compiler work through configured lanes.	Raw generated text remains untrusted; configured independent checks are required.
Adaptive proof heads	Predict route availability, outcome, premise family, reconstruction, and failure categories.	Version-matched admitted feedback; prediction cannot replace checking.

735 O.1 Planning is deterministic until explicitly guided

736 The local `LogicTactician` validates the goal and policy, binds the configuration identity, validates
737 and partitions candidate sources, optionally applies pinned guidance, selects routes under budgets, and
738 decomposes declared proof gaps. Cycles cause abstention rather than a malformed plan. Excluded
739 sources retain a reason, and final plans carry input roots and guidance identities. The planner’s own
740 implementation performs no proof, write, or network work. [artifact S38]

741 This is a domain-neutral orchestration layer. Its name should not be taken to imply implementation of
742 an external tactic-learning system. A separate executor is needed to realize a plan, and that execution
743 must supply its own artifacts and outcomes. Optional ranking and nomination have a narrower
744 contract than unconstrained generation of new assumptions or arbitrary executable tasks.

745 O.2 The supported-fragment Hammer workflow

746 The inspected Hammer guide describes a native goal, premise selection, translation to TPTP or SMT-
747 LIB, a bounded external portfolio, candidate normalization, reconstruction, kernel checking, and
748 receipt persistence. Each target ITP has a frontend and matching reconstructor. The first-order-safe
749 fragment admits explicit term/connective forms and finite sort signatures under the translation contract;
750 unsupported dependent types, higher-order predicate quantification, unresolved polymorphism, and
751 opaque unrecognized terms must return unsupported rather than be erased. [artifact S37]

752 All external solvers require operator opt-in. A missing native toolchain remains unavailable; a timeout
753 remains a timeout. In this ITP Hammer contract, a solver’s answer is an untrusted candidate until the
754 originating native checker accepts reconstruction. This statement is specific to that contract; another

755 SMT-facing API may report a solver-local result without representing it as a native ITP theorem.
756 [artifact S19,S37]

757 A theorem applies to the formalized premises and goal. It does not independently establish that the
758 original prose, source-code extractor, or cross-domain identity mapping was correct. In particular,
759 a valid reconstructed proof of an incorrectly simplified obligation does not repair the semantic loss
760 caused by the simplification.

761 **O.3 What the optional premise selector actually learns**

762 The deterministic baseline uses symbol, type, import, and graph-overlap features. The optional
763 selector artifact applies a bounded linear score followed by a sigmoid, with additional count features
764 and a pinned model digest. The supplied default weights are hand-authored. Its documentation
765 explicitly does not claim a trained neural or tree-based checkpoint. A genuinely learned artifact can
766 fit the same interface, but the training dataset, loss, split, parameter provenance, and evaluation would
767 then require separate evidence. [artifact S36]

768 The documented held-out benchmark uses shared-import proximity as a relevance proxy because
769 the fixture corpus lacks explicit theorem-to-theorem dependency labels. Recall or reciprocal rank
770 against that proxy is not ground-truth proof-premise recall. A digest binds the loaded artifact; it
771 neither demonstrates training nor proves that the source of the digest was independently trusted.

772 **O.4 Leanstral does not train itself by generating a draft**

773 The security proof-assistant lane treats Leanstral output as advisory and checks the proposed Lean
774 code against a designated proof consumer. Its policy rejects proof bypasses such as `sorry`, `admit`,
775 or unapproved axioms. Configuration readiness, an actual model call, a generated candidate, and a
776 checker-accepted proof are different events. The source audit does not demonstrate fine-tuning of
777 Leanstral’s base weights. [artifact S39]

778 The legal rollout gives the return path to learning an explicit artifact boundary. Candidate JSON or
779 free-form drafts are audit inputs only. Structured guidance that passes the required syntax, provenance,
780 anti-copy, proof, and configured reconstruction checks may enter the trusted feature bus and bounded
781 autoencoder training. A stable source-free export remains inert until its independent promotion gate
782 accepts it. A deterministic compiler/decompiler patch is a separately versioned executable change
783 with focused tests and frozen-canary evidence. [artifact S40]

784 The documented rollout is an ordered smoke, hyperparameter-selection, fixed-canary, and production
785 sequence. Promotion checks per-family semantics, reconstruction, source-copy behavior, trusted-
786 feedback receipt agreement, process outcomes, queue telemetry, accepted-patch rate, and rollback
787 identity. These are documented control requirements, not evidence that this manuscript executed or
788 completed such a rollout.

789 **P Putting the representations and update loops together**

790 This explanatory procedure composes inspected interfaces while explicitly separating design from
791 execution. It is not a claim that one repository function implements the entire procedure with every
792 optional component enabled.

793 **Construct and ground.** Begin with a versioned source and provenance. Retrieve candidate frames
794 or context through the configured lexical, numerical, or graph route. Compile an interpretation using
795 the chosen domain adapter and record ambiguity or unsupported semantics. Preserve the canonical
796 declaration independently of any learned feature vector. Numerical targets identify their producer;
797 compiler-derived targets identify their teacher. [artifact S01,S07,S22,S34,S35]

798 **Project and challenge.** Produce named logical views using shared symbols and explicit projection
799 contracts. A frame view records roles, a deontic view records normative force, a temporal view
800 records clock relations, and an event view records transitions. Source-withheld decompilation exposes
801 missing reconstructable information. Minimal pairs and independent annotations challenge omissions
802 that a compiler/decompiler cycle could share. [artifact S04,S07,S09,S31]

803 **Plan, propose, and check.** A query generates obligations under the relevant interpretation. The
804 Tactician selects bounded routes from admitted candidates. A Hammer can search a supported
805 translation and attempt native reconstruction; Leanstrat may instead suggest or repair a candidate in an
806 approved lane. Record failure, counterexample, timeout, unsupported translation, and unavailability
807 distinctly. Only the actual checker and its pinned inputs establish the scope of a checked result.
808 [artifact S37–S39]

809 **Improve the right component.** An embedding residual or family/view classification defect can
810 motivate a shared-parameter update. A repeated loss of exception scope can motivate a com-
811 piler/decompiler code change. A poorly selected proof route can motivate an isolated proof-head
812 update, using only admitted eligible feedback. These diagnoses are examples of how to use the avail-
813 able signals, not measured examples of autonomous learning. Teacher fidelity, primary representation
814 quality, proof-head calibration, and executable repair correctness remain different objectives. [artifact
815 S24–S30,S40]

816 **Promote and reuse.** Freeze the accepted model or executable change, its targets, data split, con-
817 figuration, and evidence. Source-free guidance requires independent fixed-canary promotion and a
818 receipt that the evaluated consumer actually loaded it. Reuse exact artifacts with their dependency
819 identities, not merely similar vectors. Keep untouched test data separate from model-selection and
820 canary populations. [artifact S01,S24,S30,S40]

821 **P.1 Legal, Intent, and Security IR composition**

822 For the main paper’s protected-write example, a Legal declaration can express an applicable constraint,
823 an Intent declaration can describe a plan to request approval and write, and a Security declaration
824 can model whether the implementation enforces the check. An event/trace view can then describe
825 observed requests and writes. A shared name is not sufficient to join them: actor, resource, tenant,
826 operation, time, and policy-version mappings must be admitted. [artifact S01,S04,S09]

827 The plan’s postcondition is not evidence that it occurred. A permission in an intent or policy view is
828 not a record of execution. A proof that a model enforces a guard is not a legal-applicability decision.
829 The useful common space is therefore a collection of linked typed declarations and evidence records,
830 with explicit bridge obligations, rather than one blended verdict or universal latent representation.

831 A learned view head can recommend examining temporal structure, but the temporal bridge must
832 still specify its clock and scope. A graph can locate a candidate exemption provision, but the source
833 interpretation must establish applicability. A counterexample can reveal a model defect, but a changed
834 compiler must not remove the difficult premise merely to make the theorem provable. These are the
835 practical reasons for keeping retrieval, semantic interpretation, statistical guidance, and checking
836 separate while permitting structured feedback among them.

Q Additional retrieval and proof-assistance evaluation

The training arms T0–T5 remain unchanged. Retrieval and proof-assistance conditions should be evaluated on the same frozen tasks, with controlled budgets, rather than folded into an unexplained end-to-end score. The following are proposed experiments; no outcomes are claimed in this revision.

Table 13: Separate measurements for retrieval, planning, and proof assistance.

Comparison	Controlled variable	Required outcome
Lexical / vector / graph	Same corpus, queries, target interpretation and budget	Independently judged relevant context and accepted-premise yield
Baseline / graph selector	Same goal and candidate corpus	Recall against stated labels; proxy status; actual useful proof coverage
Unguided / guided plan	Same admitted sources and route budget	Deterministic replay, subgoal coverage, abstention and total cost
Hammer / Leanstral assistance	Same source goal, native checker and resource envelope	Candidate rate, reconstruction acceptance, unsupported and timeout rates
Proof heads off / on	Same representation checkpoint and isolated update policy	Calibration, route value, protected-objective invariance
Guidance off / promoted	Same compiler profile and frozen source families	Actual consumer activation, independent source fidelity, regressions

For all conditions, retain the corpus and index revision, tokenizer, embedding producer, index implementation, model digest, admitted premises, goal root, logical profile, supported translation fragment, toolchain, checker, time/memory limits, and outcome. Record fixture or mock targets explicitly. Count unsupported, unavailable, and abstained cases in coverage denominators rather than reporting only successful requests. End-to-end cost includes target construction, indexing, gradient updates, model calls, failed candidates, proof reconstruction, semantic validation, and human review.

These experiments distinguish three hypotheses: better access to relevant context; better search under a fixed formal interpretation; and better fidelity of that interpretation to the source. Improvements in one do not establish improvements in the other two. No new performance value, trained checkpoint, native solver execution, or deployment qualification is inferred from rebuilding this paper.

851 **Paper checklist: official questionnaire required**

852 The uploaded workshop research template includes `checklist.tex`, but that questionnaire was not
853 included in the supplied sources. This page is a compilation placeholder, not an official questionnaire
854 and not a set of completed compliance answers.

855 **[TODO]** Replace this file with the official checklist and complete its responses against the final
856 manuscript and artifact.

857 **[TODO]** The draft still needs author review, a frozen anonymous implementation artifact, and the
858 stated empirical evaluation or an explicitly narrowed non-empirical submission claim. No experiment
859 or checklist answer is inferred from a successful LaTeX build.